

Connected Components Pattern (DFS/BFS) - Complete Notes

One of the most important Graph/Grid patterns for coding interviews.

Master this pattern and you'll be able to solve **20-30+ LeetCode grid problems** easily.

Table of Contents

1. What is a Connected Component?
 2. Why do we need Connected Components?
 3. How to Identify the Pattern
 4. DFS/BFS Traversal Idea
 5. Generic Algorithm
 6. Universal DFS Template
 7. Universal BFS Template
 8. Common Variations
 9. Difference Between Connected Components & Multi-Source DFS/BFS
 10. Time & Space Complexity
 11. Problems to Practice
 12. Recognition Checklist
-

1. What is a Connected Component?

A connected component is:

A group of connected cells/nodes where every node can reach every other node within the group.

For grid problems, connection usually means:

- Up
- Down
- Left
- Right

Sometimes diagonal connections are also considered (depends on the problem).

Example

```
1 1 0 0
1 0 0 1
```

```
0 0 1 1
1 0 0 0
```

Visual representation:

```
■ ■ □ □
■ □ □ ■
□ □ ■ ■
■ □ □ □
```

There are **3 connected components**.

Component 1

```
■ ■
■
```

Component 2

```
  ■
■ ■
```

Component 3

```
■
```

2. Why do we need Connected Components?

Suppose we only run

```
dfs(0,0)
```

It only explores

```
■ ■
■
```

The remaining islands are never visited.

Therefore we must scan the entire grid.

```
for every cell
    if new component found
        run DFS/BFS
```

Every DFS/BFS discovers **one complete connected component**.

3. How to Identify Connected Components Problems

These keywords usually indicate this pattern.

Keywords

- Island
 - Region
 - Cluster
 - Group
 - Province
 - Connected
 - Area
 - Component
-

Questions usually ask

Count something

```
How many islands?

How many provinces?

How many groups?
```

Find largest/smallest

```
Largest island

Maximum area

Maximum fish
```

Largest region

Calculate value of every component

Sum

Area

Perimeter

Weight

Population

If the problem says

Find every disconnected group

or

Process each island separately

It is almost always Connected Components.

4. DFS Traversal Idea

Imagine

```
1 1 0
1 0 1
```

Start scanning.

Found

$(0,0)$

Run DFS

$(0,0)$

```
↓  
  
(0,1)  
  
↓  
  
(1,0)
```

DFS finishes.

Entire island visited.

Continue scanning.

Eventually find

```
(1,2)
```

Run DFS again.

Repeat until grid ends.

5. Generic Algorithm

Step 1

Create visited array.

```
visited[][] = False
```

Step 2

Scan every cell.

```
for every row  
    for every column
```

Step 3

If

- valid cell

- not visited

then

DFS/BFS

Step 4

DFS explores entire component.

Step 5

Use returned information.

Examples

```
count++  
  
max(area)  
  
sum  
  
perimeter  
  
etc.
```

6. Universal DFS Template

```
visited = [[False]*cols for _ in range(rows)]  
  
directions = [  
    (0,1),  
    (0,-1),  
    (1,0),  
    (-1,0)  
]  
  
def dfs(r,c):  
    visited[r][c] = True  
    value = contribution_of_current_cell  
    for dr,dc in directions:
```

```
        nr = r + dr
        nc = c + dc

        if valid neighbour:

            value += dfs(nr,nc)

    return value

answer = initial_value

for every cell:

    if starting cell and not visited:

        component_value = dfs(i,j)

        process(component_value)

return answer
```

7. Universal BFS Template

```
from collections import deque

queue = deque()

visited = ...

queue.append(start)

visited[start] = True

while queue:

    r,c = queue.popleft()

    for every neighbour:

        if valid:

            visited = True

            queue.append(neighbour)
```

```
for every cell:
    if not visited:
        bfs()
```

Exactly same pattern.

Only traversal changes.

8. Common Variations

Variation 1

Count Components

Example

Number of Islands

DFS returns nothing.

```
count += 1
```

Variation 2

Maximum Area

DFS returns

```
number of cells
```

Example

```
■ ■
■
```

returns

3

Answer

`max(area)`

Variation 3

Maximum Sum

Example

2 3

4

DFS returns

9

Answer

`max(sum)`

Used in

Maximum Number of Fish

Variation 4

Component Sum Condition

DFS returns

sum

Then

```
if sum % k == 0  
  
count++
```

Variation 5

Perimeter

DFS returns

```
perimeter contribution
```

Answer

```
total perimeter
```

9. Connected Components vs Multi-Source DFS/BFS

Connected Components

We DON'T know where components start.

We discover them while scanning.

```
for every cell  
  
    if new island  
  
        dfs()
```

Examples

- Number of Islands
- Max Area
- Max Fish
- Count Islands

Multi-Source DFS

We ALREADY know the starting points.

Run DFS from every source.

Example

Surrounded Regions

Sources

```
Boundary 0's
```

```
for every boundary 0
    dfs()
```

Goal

Mark all safe cells.

Multi-Source BFS

Queue initially contains ALL sources.

Example

Rotten Oranges

```
2 1 1
1 1 0
0 1 2
```

Queue

```
[ (0,0), (2,2) ]
```

Both spread simultaneously.

10. Time Complexity

Suppose

Rows = R

Columns = C

Every cell visited exactly once.

Time = $O(R \times C)$

Visited array

Space = $O(R \times C)$

DFS recursion

Worst case

$O(R \times C)$

11. Practice Problems

Easy

✓ Number of Islands (LC 200)

Learn counting connected components.

✓ Max Area of Island (LC 695)

Learn returning area.

✓ Flood Fill (LC 733)

Basic DFS/BFS.

Medium

✓ Maximum Number of Fish in a Grid (LC 2658)

Return component sum.

✓ Count Sub Islands (LC 1905)

Compare two grids.

✓ Number of Provinces (LC 547)

Connected components in graph.

✓ Number of Enclaves (LC 1020)

Boundary DFS.

✓ Surrounded Regions (LC 130)

Multi-source DFS.

✓ Closed Islands (LC 1254)

Connected components + boundary condition.

Hard

✓ Making A Large Island (LC 827)

Label connected components.

✓ Shortest Bridge (LC 934)

Connected Components + Multi-source BFS.

12. Recognition Checklist

If the question asks

- ✓ Count islands
- ✓ Count groups
- ✓ Count provinces
- ✓ Largest island
- ✓ Largest region
- ✓ Area of component
- ✓ Sum of component
- ✓ Perimeter
- ✓ Process every disconnected group

Then think

Connected Components DFS/BFS

If the question says

- ☒ Start from boundary
- ☒ Start from every gate
- ☒ Start from every rotten orange
- ☒ Start from every ocean
- ☒ Start from every source

Then think

Multi-Source DFS/BFS

Master Template (Remember This)

Scan entire grid

↓

Found new unvisited valid cell?

↓

YES

↓

Run DFS/BFS

↓

Explore ENTIRE connected component

↓

Return component information

↓

Update answer

↓

Continue scanning

↓

Done

Interview Mindset

Whenever you see a grid problem, ask these questions in order:

Question 1

Do I need to visit every connected group separately?

→ Connected Components

Question 2

Do I already know all starting points?

→ Multi-Source DFS/BFS

Question 3

Am I exploring every possible path?

→ Backtracking

Question 4

Do I need the shortest path?

→ BFS

Question 5

Do I simply need to explore one connected region?

→ Single DFS/BFS

This decision tree alone can help identify the correct traversal pattern in most graph and grid interview problems.